T.C. AYDIN ADNAN MENDERES ÜNİVERSİTESİ

Aydın Meslek Yüksekokulu - Bilgisayar Programcılığı Programı

WEB PROGRAMCILIĞI III (BPR201)

1. Hafta: Git ve GitHub Araştırma ve Uygulama Ödevi

Öğrenci Adı Soyadı:	Volkan KAYA
Öğrenci Numarası:	256001075 (256001075@stu.adu.edu.tr)
Öğretim Görevlisi:	Öğr. Gör. Mehmet Can Hanaylı
Dönem:	2026-2027 Güz Yarıyılı

---

1. ARAŞTIRMA BÖLÜMÜ: GİT VE GİTHUB KAVRAMLARI

Bu bölümde derste ve ödev föyünde verilen Git ve GitHub kavramlarını araştırdım. Yazılım geliştirirken sürüm kontrolünün neden önemli olduğunu ve hangi sorunları çözdüğünü kendi cümlelerimle açıkladım.

- Git nedir? Git, yazdığımız kodların zaman içindeki tüm değişimlerini kaydeden açık kaynaklı bir sürüm kontrol sistemidir. Linus Torvalds tarafından geliştirilmiştir. En büyük özelliği dağıtık (distributed) yapıda olmasıdır; yani internete bağlı olmadan da kendi bilgisayarımızda tüm geçmişi tutarak çalışabiliriz.

- GitHub nedir? GitHub, Git ile takip ettiğimiz projelerimizi internet üzerinde saklamamıza, yedeklememize ve diğer insanlarla birlikte geliştirmemize yarayan bulut tabanlı bir servistir. Ayrıca hata takibi (Issue), kod birleştirme istekleri (Pull Request) ve otomatik işlemler (GitHub Actions) gibi imkanlar sağlar.

- Git ile GitHub arasındaki fark nedir? Git bilgisayarımıza kurduğumuz bir komut satırı programıdır, GitHub ise bu Git depolarını internette depoladığımız web platformudur. Git internet olmadan da çalışır, GitHub ise projeyi başkalarıyla paylaşmak ve uzakta yedeklemek için internet gerektirir.

- Repository (Depo) nedir? Bir projeye ait bütün kaynak dosyaların, klasörlerin ve yapılan bütün değişiklik geçmişinin (commit kayıtlarının) tutulduğu ana dizindir. Proje klasöründeki gizli .git klasörü o projenin Git deposu olduğunu gösterir.

- Commit nedir? Projeye yaptığımız bir değişikliğin kalıcı olarak kaydedilmesidir. Her commit'in kendine ait benzersiz bir kimlik kodu (hash), yazarı, tarihi ve ne yapıldığını anlatan bir açıklama mesajı bulunur.

- Branch (Dal) nedir? Ana kod tabanını (main) bozmadan yeni bir özellik geliştirmek veya bir hatayı denemek için açılan ayrı çalışma yoludur. Böylece yeni yazdığımız kodlar hazır olana kadar ana çalışan proje zarar görmez.

- Merge nedir? Farklı bir dalda geliştirip tamamladığımız kodları hedef dala (genellikle main dalına) aktararak iki dalı birleştirme işlemidir.

- Clone nedir? İnternette (örneğin GitHub üzerinde) bulunan bir projeyi bütün geçmişiyle birlikte kendi bilgisayarımıza indirme işlemidir (git clone komutu ile yapılır).

- Pull nedir? Uzak depoda (GitHub) yapılan en güncel değişiklikleri kendi bilgisayarımıza indirip yerel çalışma dalımızla birleştirme işlemidir (git pull komutu ile yapılır).

- Push nedir? Kendi bilgisayarımızda yaptığımız commit'leri uzak depoya (GitHub'a) yükleme işlemidir (git push komutu ile yapılır).

- .gitignore dosyasının amacı nedir? Git'in takip etmesini istemediğimiz dosyaları belirttiğimiz ayar dosyasıdır. Örneğin şifrelerin bulunduğu .env dosyaları, işletim sistemi artıkları (Thumbs.db, .DS_Store), editör ayarları (.vscode) veya gereksiz büyük kütüphane klasörleri (node_modules) bu dosyaya yazılarak Git'e yüklenmesi engellenir.

Git Hangi Problemleri Çözer ve Web Projelerinde Neden Kullanılır?

Eskiden yazılımcılar yaptıkları değişiklikleri saklamak için klasörleri 'proje_son.zip', 'proje_final2.zip' gibi isimlerle kopyalıyordu. Git bu karmaşayı tamamen çözer; kimin hangi satırı ne zaman değiştirdiğini gösterir. Ayrıca yazılan kodda bir hata çıkarsa sistemi bozmadan çalışan eski bir commit'e anında geri dönebiliriz. Ekip çalışmalarında herkesin aynı dosyalar üzerinde çakışmadan çalışabilmesini sağlar. Alternatif Teknolojiler: Git dışında geçmişte SVN (Subversion) ve Mercurial gibi sistemler kullanılmıştır, ancak günümüzde sektörün büyük çoğunluğu Git kullanmaktadır. GitHub alternatifi olarak ise GitLab ve Bitbucket platformları bulunmaktadır.

2. SEKTÖR ARAŞTIRMASI: 3 AÇIK KAYNAK WEB PROJESİ

GitHub üzerinde yaygın kullanılan 3 açık kaynak web projesini inceledim. Projelerin amaçları, dilleri, klasör ve README düzenleri aşağıda özetlenmiştir:

1. Proje: Next.js (Vercel)

GitHub Adresi:	https://github.com/vercel/next.js
Kullanılan Dil ve Framework:	TypeScript, JavaScript / React tabanlı Fullstack Web Framework
Projenin Amacı:	React ile hızlı, SEO uyumlu, sunucu taraflı çalışan web uygulamaları geliştirmek.
Commit ve Katkıcı Sayısı:	~35.000 commit / 3.600+ katkıcı
Klasör Yapısı:	packages/ (çekirdek kütüphaneler), examples/ (örnek şablonlar), test/ (test kodları).
README Düzeni:	Kurulum adımları (npx create-next-app), rozetler ve kapsamlı dokümantasyon linkleri yer alır.

2. Proje: Laravel

GitHub Adresi:	https://github.com/laravel/laravel
Kullanılan Dil ve Framework:	PHP / Laravel Web Framework (MVC Mimarisi)
Projenin Amacı:	Temiz ve okunabilir bir sözdizimi ile kurumsal ölçekte MVC web uygulamaları geliştirmek.
Commit ve Katkıcı Sayısı:	~42.000 commit (framework dahil) / 3.400+ katkıcı
Klasör Yapısı:	app/ (Controller ve Modeller), routes/ (web ve api rotaları), resources/ (Blade arayüz şablonları).
README Düzeni:	Ekosistem araçları (Forge, Vapor), sponsorlar ve detaylı kullanım yönergeleri bulunur.

3. Proje: FastAPI

GitHub Adresi:	https://github.com/fastapi/fastapi
Kullanılan Dil ve Framework:	Python / Starlette ve Pydantic tabanlı ASGI Framework
Projenin Amacı:	Modern Python ile yüksek performanslı ve otomatik Swagger dokümantasyonuna sahip REST API'lar üretmek.
Commit ve Katkıcı Sayısı:	~6.500 commit / 750+ katkıcı
Klasör Yapısı:	fastapi/ (çekirdek kodlar), docs/ (farklı dillerde çevrilmiş dokümantasyon), tests/ (testler).
README Düzeni:	Çok dilli çeviriler, çalışan kod örnekleri ve interaktif Swagger arayüzü anlatımları bulunur.

3. UYGULAMA ÖDEVİ: TARIM VE MAHSUL TAKİP PORTALI

Dönemlik projemiz kapsamında Aydın yöresi üreticileri için 'Tarım Kooperatifi ve Mahsul Takip Portalı' isimli bir web uygulaması hazırladım. Uygulamada HTML, CSS ve JavaScript kullanılmıştır. Sayfalar sade, kullanışlı ve responsive (mobil uyumlu) olarak tasarlanmıştır:
1. index.html: Ana sayfa. Aydın Efeler istasyonuna ait toprak nemi (%44.2), sıcaklık (24.5 °C) ve vana durumunu özet olarak gösterir.
2. mahsuller.html: Çiftçilerin tarladaki mahsulleri (İncir, Pamuk, Zeytin, Mısır vb.) filtreleyebildiği, arama yapabildiği ve canlı vana açma/kapatma kumandasının bulunduğu takip panelidir. Ayrıca listeye yeni mahsul eklenebilmektedir.
3. iletisim.html: Kooperatif kaydı ve toprak nem sensörü talep formu ile iletişim bilgilerinin yer aldığı sayfadır.
4. css/style.css: Tek ana renk tonunda (koyu yeşil/gri), göz yormayan sade tasarım kuralları tanımlanmıştır.
5. js/app.js: Canlı saat göstergesi, sensör dalgalanması simülasyonu, vana aç/kapa mantığı ve filtreleme kodları çalışmaktadır.

4. GİT SÜRÜM KONTROLÜ VE ANLAMLI COMMIT GEÇMİŞİ

Ödev föyündeki 'en az 5 anlamlı commit bulunmalıdır' şartına uygun olarak, projede 'commit1', 'deneme' gibi rastgele mesajlar yerine her yapılan işi açıklayan 9 adet sıralı commit atılmıştır:

No	Commit Mesajı	Açıklama
1	`Initial project structure and repository configuration`	Proje ana klasörü, .gitignore ve kök README.md dosyası oluşturuldu.
2	`Add hafta-01 web application core structure and styles`	index.html temel iskeleti ve css/style.css dosyası eklendi.
3	`Add responsive navigation menu and crop dashboard components`	mahsuller.html sayfası ve mahsul takip tablosu tasarlandı.

4	**Add dynamic telemetry simulation and interactive form handling**	js/app.js ile sensör değerleri ve vana kumandası kodlandı.
5	**Add contact page and cooperative communication interface**	iletisim.html çiftçi başvuru formu oluşturuldu.
6	**Update project documentation and research notes**	Hafta 1 detaylı README.md dokümanı tamamlandı.
7	**Enhance UI layout, market tickers, weather widgets and live search filtering**	Tablo içi filtreleme ve arama özellikleri eklendi.
8	**Add dynamic crop management, interactive FAQ accordion and extended telemetry metrics**	Dinamik mahsul ekleme formu ve istasyon seçici eklendi.
9	**Simplify design and user interface to clean monochrome layout**	Tasarım sadeleştirildi, göz yormayan tek renk şablonuna çekildi.

5. KARŞILAŞILAN PROBLEMLER, ÖĞRENİLENLER VE KAYNAKLAR

Karşılaşılan Problemler ve Çözümleri:

1. .gitignore dosyasını ilk başta eklemeyi unuttuğum için işletim sisteminin oluşturduğu bazı geçici dosyalar Git takibine girmişti. Daha sonra .gitignore eklesem de dosyalar takipte kaldı. 'git rm --cached <dosya>' komutunu araştırıp kullanarak bu dosyaları takip havuzundan güvenle çıkardım.
2. Farklı dallarda çalışırken aynı satırların değişmesi durumunda oluşabilecek 'merge conflict' durumunu inceledim ve çakışma etiketlerinin elle nasıl düzeltildiğini öğrendim.
3. Tek bir seferde büyük commit atmak yerine projeyi adım adım geliştirip her özellik için ayrı ve anlamlı commit mesajları yazma disiplini kazandım.

Öğrenilenler ve Kazanımlar:

Git komutlarının (init, status, add, commit, branch, merge, push, pull) mantığını ve yazılım dünyasında neden zorunlu olduğunu uygulamalı olarak pekiştirdim. Açık kaynak büyük projeleri inceleyerek düzenli klasör yapısının ve dokümantasyonun ne kadar önemli olduğunu gördüm.

Kaynaklar:

1. Chacon, S., & Straub, B. (2014). Pro Git (2nd Edition). Apress. https://git-scm.com/book/en/v2
2. Atlassian Git Tutorials & Best Practices. https://www.atlassian.com/git/tutorials
3. GitHub Docs (Getting Started with Git). https://docs.github.com/en/get-started
4. Next.js Resmi GitHub Deposu: https://github.com/vercel/next.js
5. Laravel Framework Resmi GitHub Deposu: https://github.com/laravel/laravel
6. FastAPI Resmi GitHub Deposu: https://github.com/fastapi/fastapi